BrickView: Real Estate Analytics Platform

Project report · Python, SQL (SQLite), Streamlit · Aryaman Bhatnagar

Live app: https://brickview-real-estate.streamlit.app

Code: https://github.com/aryaman2213/BrickView-Real-Estate-Analytics-Platform

21,200 Listings	**20,480** Active (unsold)	**720** Sales	**$775.8M** Sales revenue
$2.45M Avg list price	**61.6** Avg days on market	**50** Agents	**20,000** Buyer records

1. Problem and approach

Buyers, sellers and agents lack an accessible way to monitor pricing, time on market and sales performance. BrickView answers that with a SQL-backed dashboard: five raw files are cleaned, stored in a normalised SQLite database, analysed with 30 SQL queries, and served through a Streamlit app with filters, maps, charts, an agent dashboard, a SQL insights page and full CRUD on every table.

Pipeline: raw JSON / CSV → `src/clean.py` → cleaned CSVs (`data/processed/`) → `sql/schema.sql` → `db/brickview.db` → Streamlit (`app/`). One command rebuilds everything: `python pipeline.py`. 43 automated tests (`pytest`) check row counts, integrity, all 30 queries, answers against pandas, filters and CRUD.

2. Data preparation

table	rows	source file
agents	50	agents_cleaned.json
listings	21,200	listings_final_expanded.json
property_attributes	21,200	property_attributes_final_expanded.json
sales	720	sales_cleaned.csv
buyers	20,000	buyers_cleaned.json

Cleaning log

table	step	rows_affected
agents	drop duplicate agent_id	0

table	step	rows_affected
agents	standardise column names, numeric types, lower-case emails	50
listings	drop duplicate listing_id	0
listings	round price / area to 2 dp, coordinates to 6 dp; dates to YYYY-MM-DD	21,200
listings	drop rows missing id / city / type / price / date	0
listings	set agent_id NULL where the agent does not exist	0
property_attributes	drop duplicate listing_id	0
property_attributes	booleans → 0/1 integers	21,200
property_attributes	swap floor_number / total_floors where floor > total floors	255
property_attributes	drop rows whose listing_id is not in listings	0
sales	drop duplicate listing_id	0
sales	drop rows whose listing_id is not in listings	0
sales	days_on_market: noisy float → whole days from the two dates (max difference 0.030 days)	720
sales	add surrogate key sale_id (1..n, ordered by listing_id)	720
buyers	drop duplicate buyer_id	0
buyers	map raw sale_id (a listing id) → sales.sale_id; drop unmatched	0
buyers	loan_taken → 0/1; provider NULL and amount 0 when no loan	9,966

Data-quality findings

- **Buyer keys:** the raw `sale_id` column holds listing IDs; it was mapped to the real `sales.sale_id`.
- **Buyer volume:** 20,000 buyer records for 720 sales (about 28 per sale), and 93% of loans exceed the sale price, so buyer figures are read as records and loan amounts as unrealistic.
- **Coordinates** are random points across the continental US for every city; the app adds a city-summary map at real city centres.
- **Two listing populations:** only L00001-L01200 have sales ($0.1M-2M); the other 20,000 listings reach $5M and are all unsold.
- **days_on_market** equals the listing-to-sale date gap to within 0.03 days and is stored as whole days.
- 255 rows with floor number above total floors were swapped; 467 non-rented homes with tenants and 2,067 rented homes with 0 tenants are flagged.
- **Agents:** `deals_closed` is a lifetime figure (average 137) and doesn't match sales in the data (average 14).

3. Database design

Five tables in third normal form. **agents** 1→many **listings**; **listings** 1→1 **property_attributes** (UNIQUE listing_id); **listings** 1→0..1 **sales** (UNIQUE listing_id); **sales** 1→many **buyers**. Foreign keys are enforced (`PRAGMA foreign_keys = ON`) with `ON DELETE CASCADE` (and `SET NULL` for a deleted agent). CHECK constraints guard prices, ratings, dates, flags and categories.

agents

column	type	not null	primary key
agent_id	TEXT		yes
name	TEXT	yes	
phone	TEXT		
email	TEXT		
commission_rate	REAL		
deals_closed	INTEGER		
rating	REAL		
experience_years	INTEGER		
avg_closing_days	INTEGER		

listings

column	type	not null	primary key
listing_id	TEXT		yes
city	TEXT	yes	
property_type	TEXT	yes	
price	REAL	yes	
area_sqft	REAL		
date_listed	TEXT	yes	
agent_id	TEXT		
latitude	REAL		
longitude	REAL		

property_attributes

column	type	not null	primary key
attribute_id	INTEGER		yes
listing_id	TEXT	yes	
bedrooms	INTEGER		
bathrooms	INTEGER		
floor_number	INTEGER		
total_floors	INTEGER		
year_built	INTEGER		
is_rented	INTEGER	yes	

column	type	not null	primary key
tenant_count	INTEGER	yes	
furnishing_status	TEXT		
metro_distance_km	REAL		
parking_available	INTEGER	yes	
power_backup	INTEGER	yes	

sales

column	type	not null	primary key
sale_id	INTEGER		yes
listing_id	TEXT	yes	
sale_price	REAL	yes	
date_sold	TEXT	yes	
days_on_market	INTEGER		

buyers

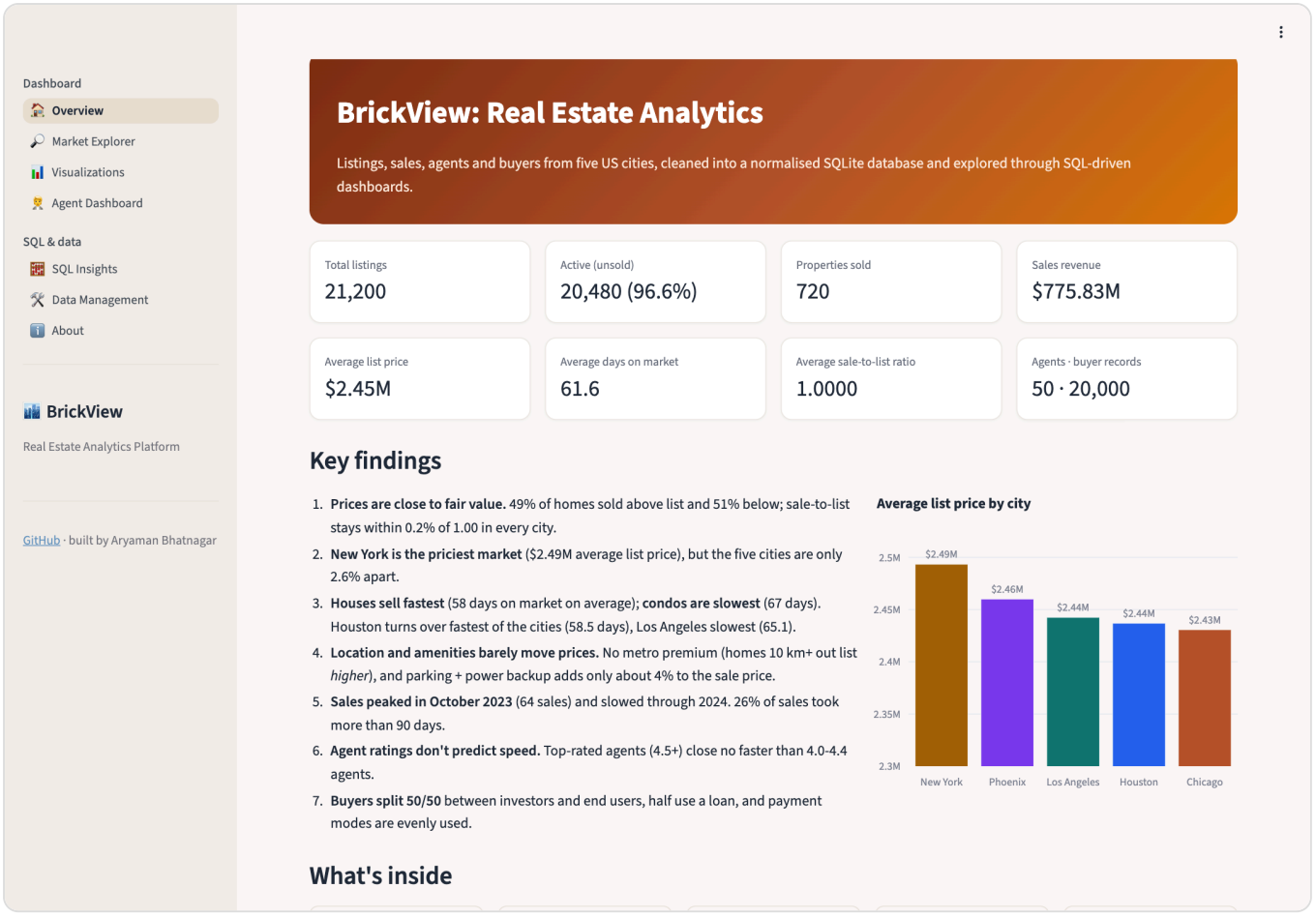
column	type	not null	primary key
buyer_id	INTEGER		yes
sale_id	INTEGER	yes	
buyer_type	TEXT		
payment_mode	TEXT		
loan_taken	INTEGER	yes	
loan_provider	TEXT		
loan_amount	REAL	yes	

Indexes and views

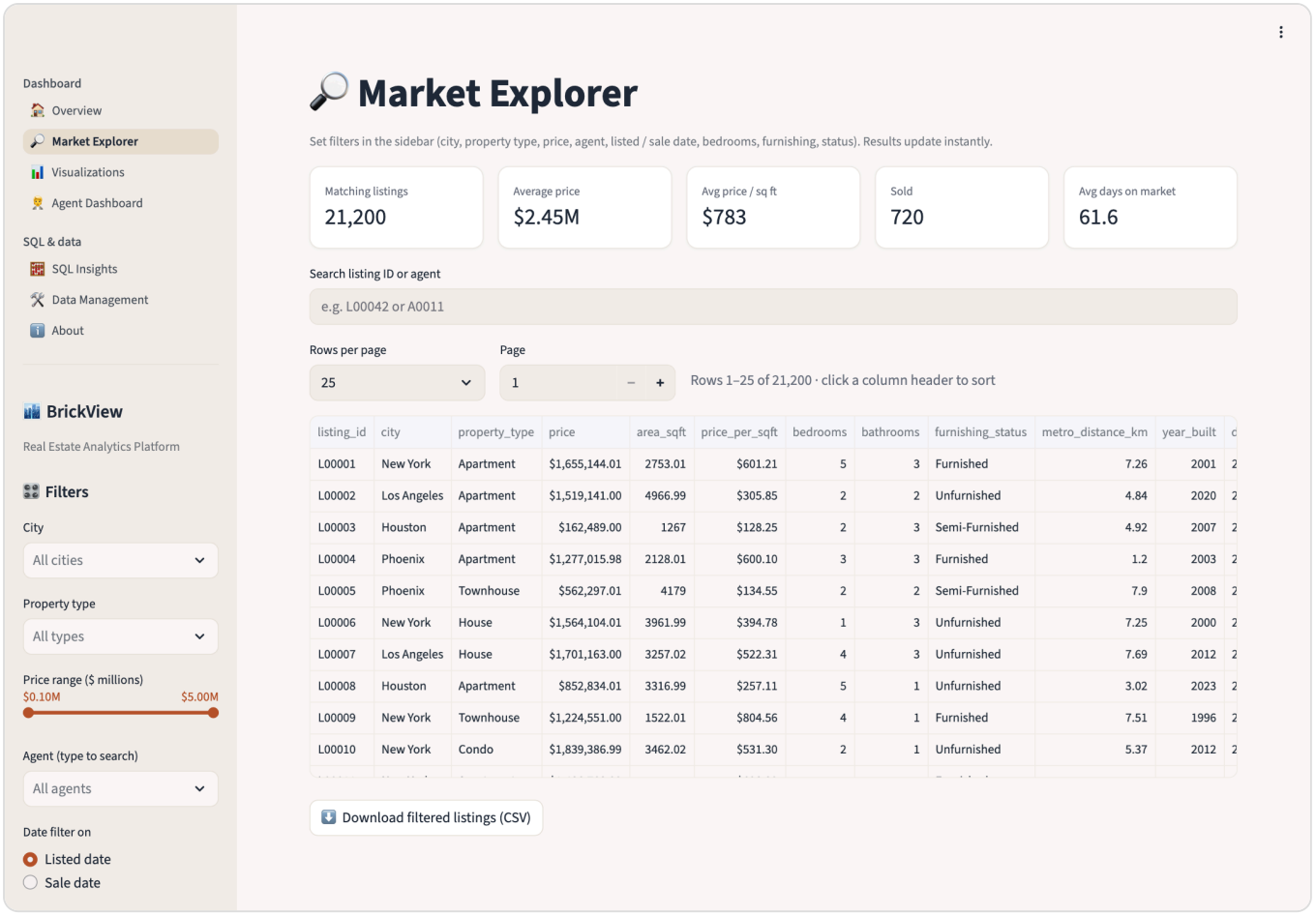
type	name
index	idx_listings_city
index	idx_listings_type
index	idx_listings_agent
index	idx_listings_price
index	idx_listings_date
index	idx_listings_city_type
index	idx_attr_furnishing
index	idx_sales_date
index	idx_buyers_sale
index	idx_buyers_type
view	v_listing_details
view	v_active_listings
view	v_sale_details
view	v_buyer_details

type	name
view	v_agent_performance
view	v_city_summary

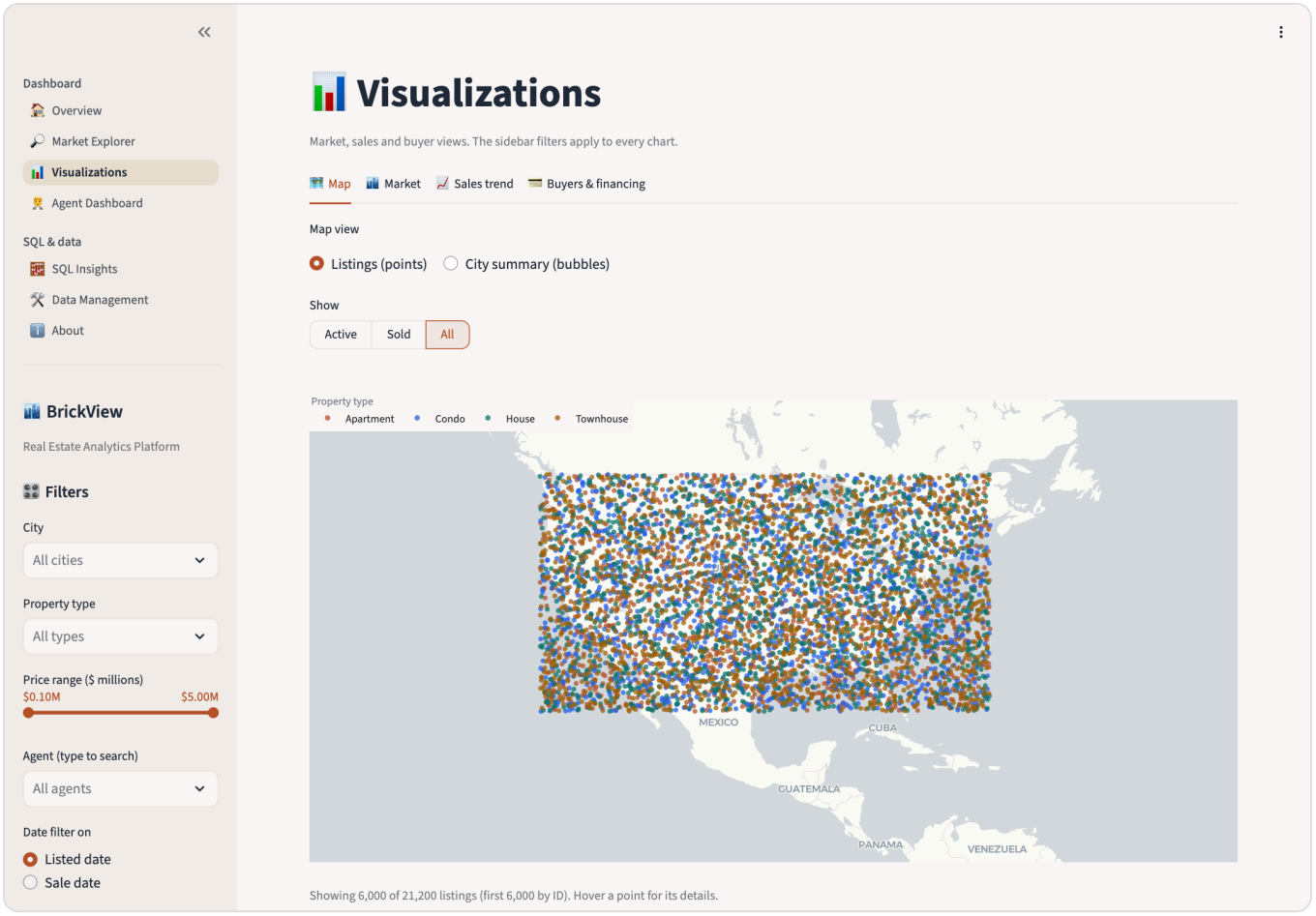
4. Streamlit application



Overview: KPIs, key findings and navigation.



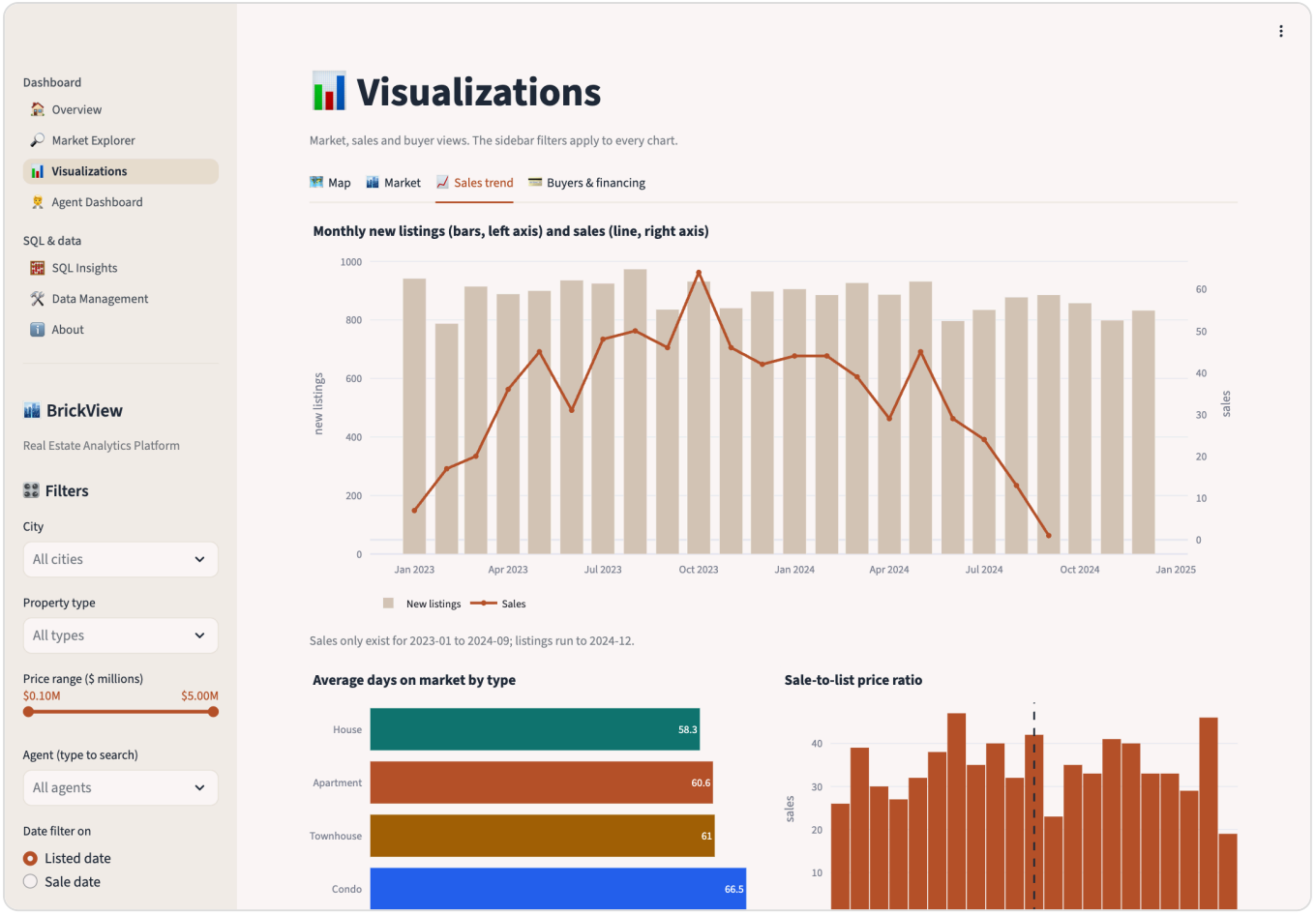
Market Explorer: sidebar filters (city, type, price, agent, listed/sale date, status, bedrooms, furnishing), paged sortable table, CSV export.



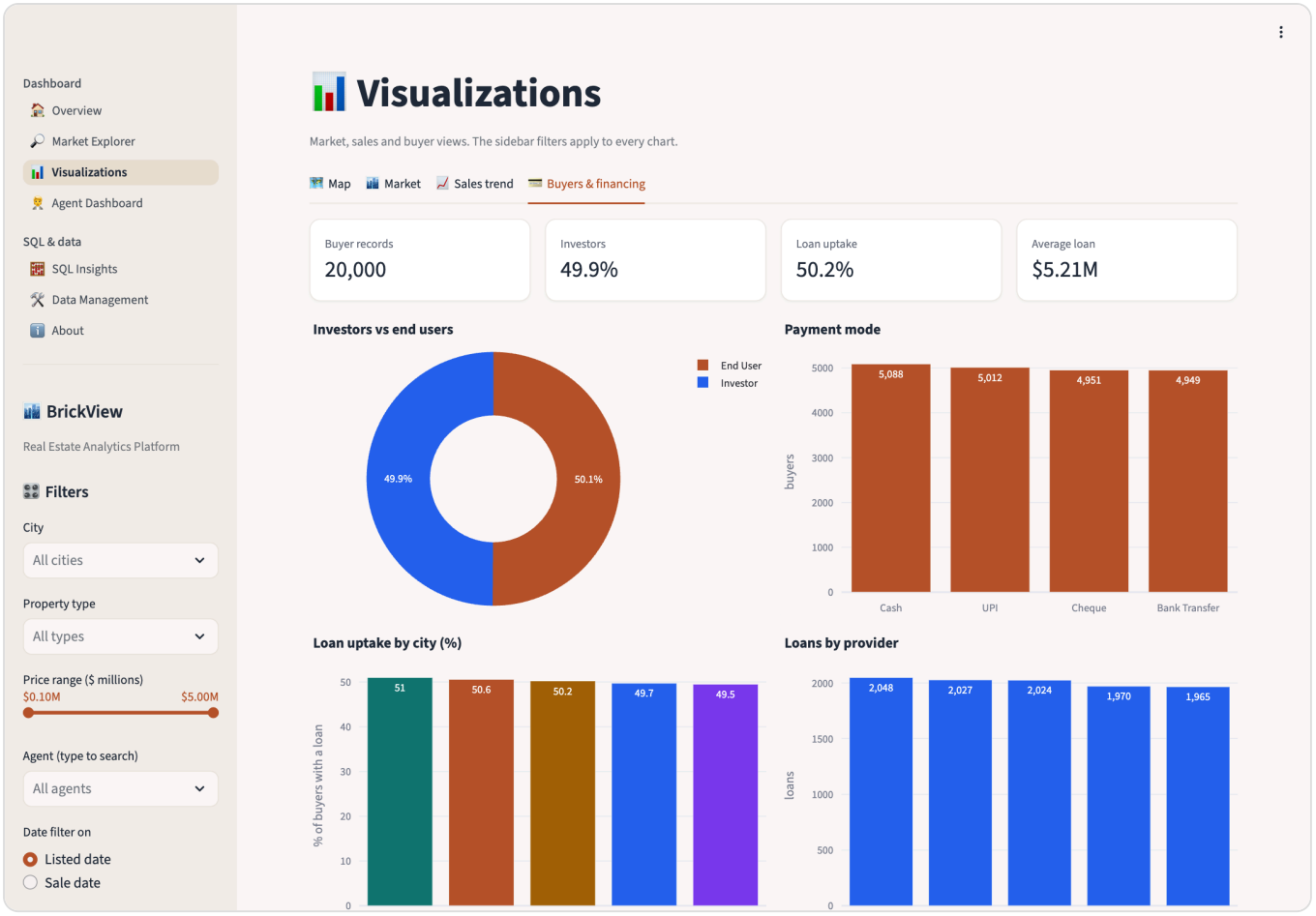
Interactive map of listings coloured by property type, with hover details; a city-summary bubble view is also available.



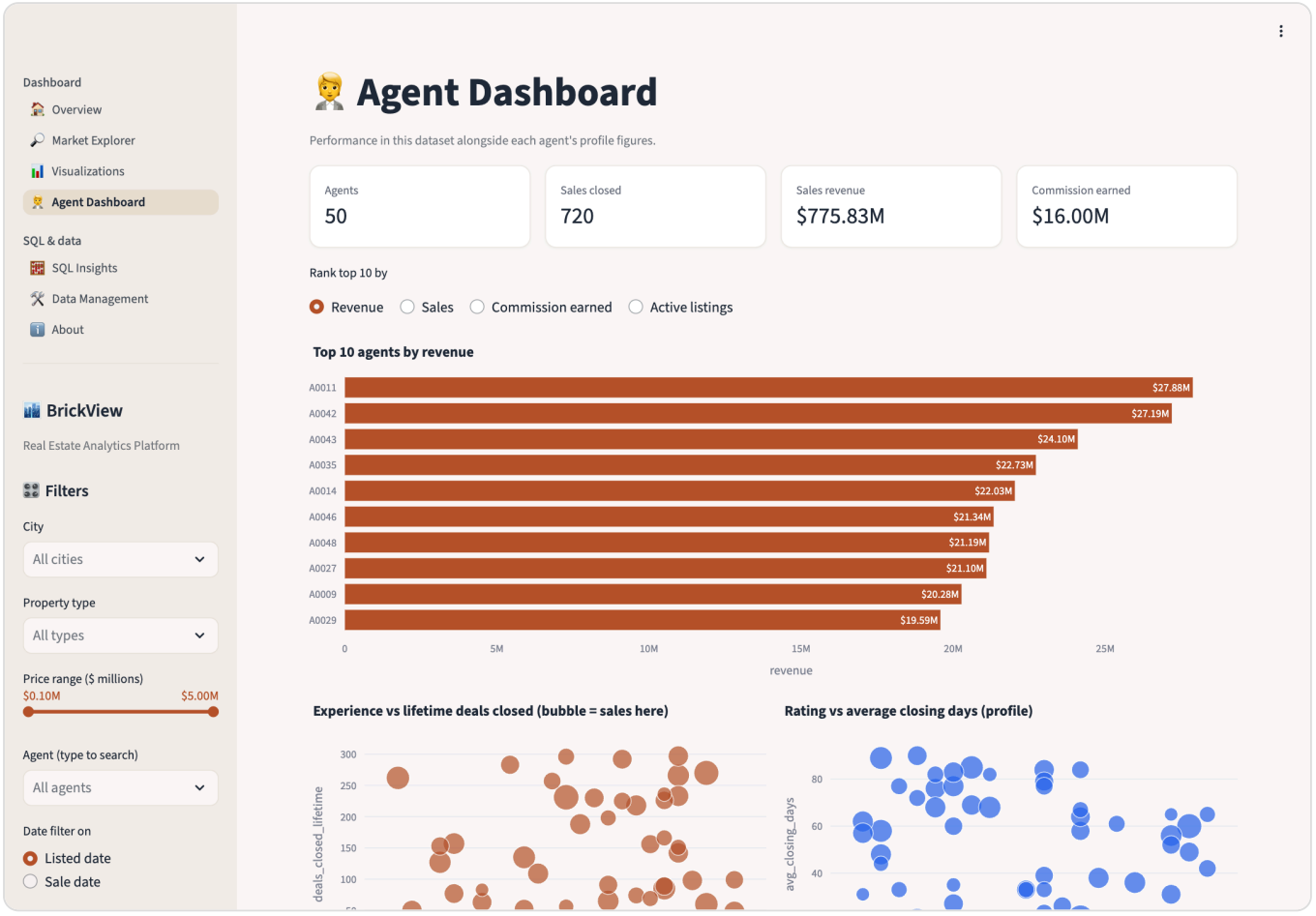
Market charts: listings / average price by city, property-type pie, price distribution, price per sq ft.



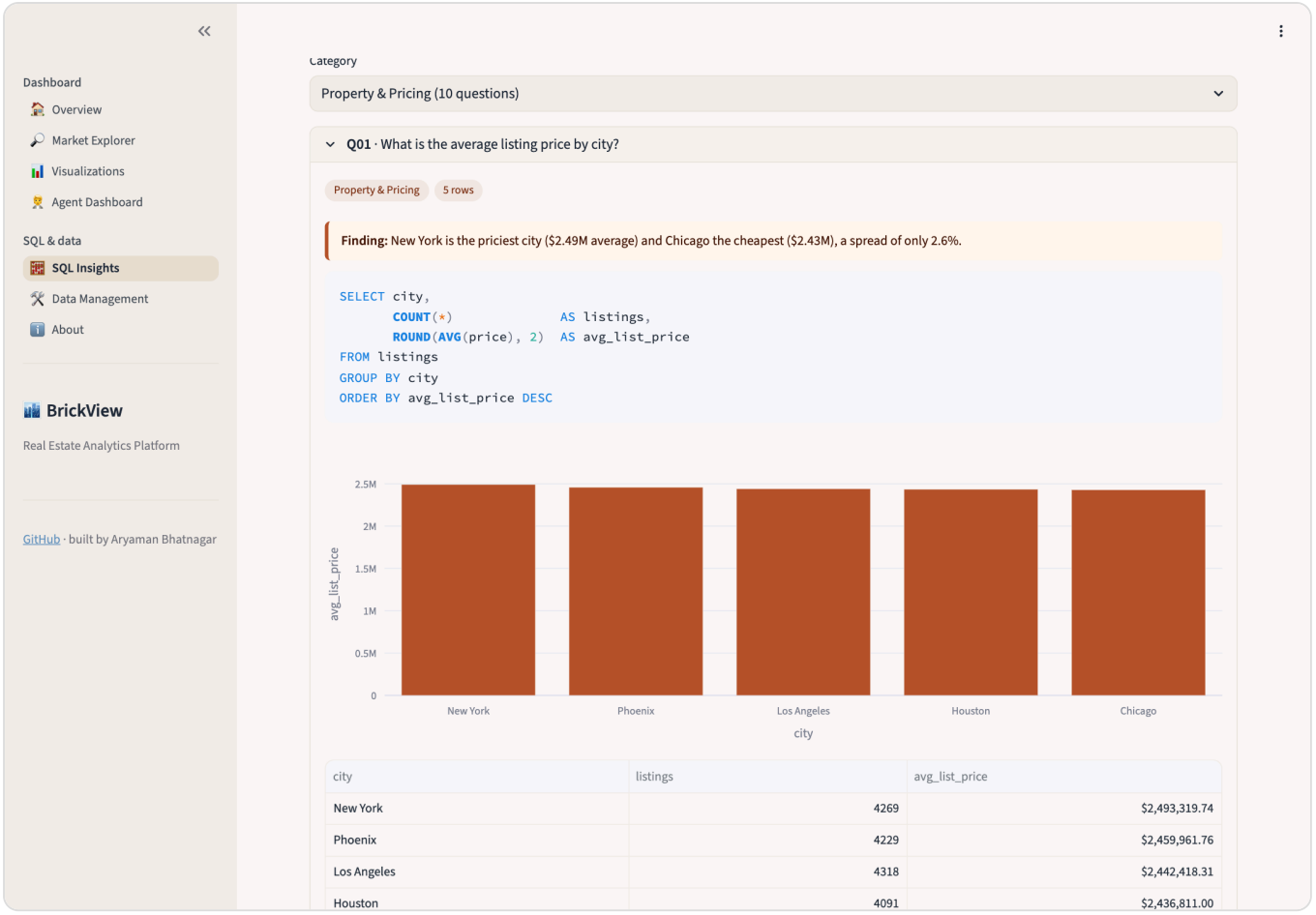
Monthly new listings vs sales, days on market by type, sale-to-list distribution.



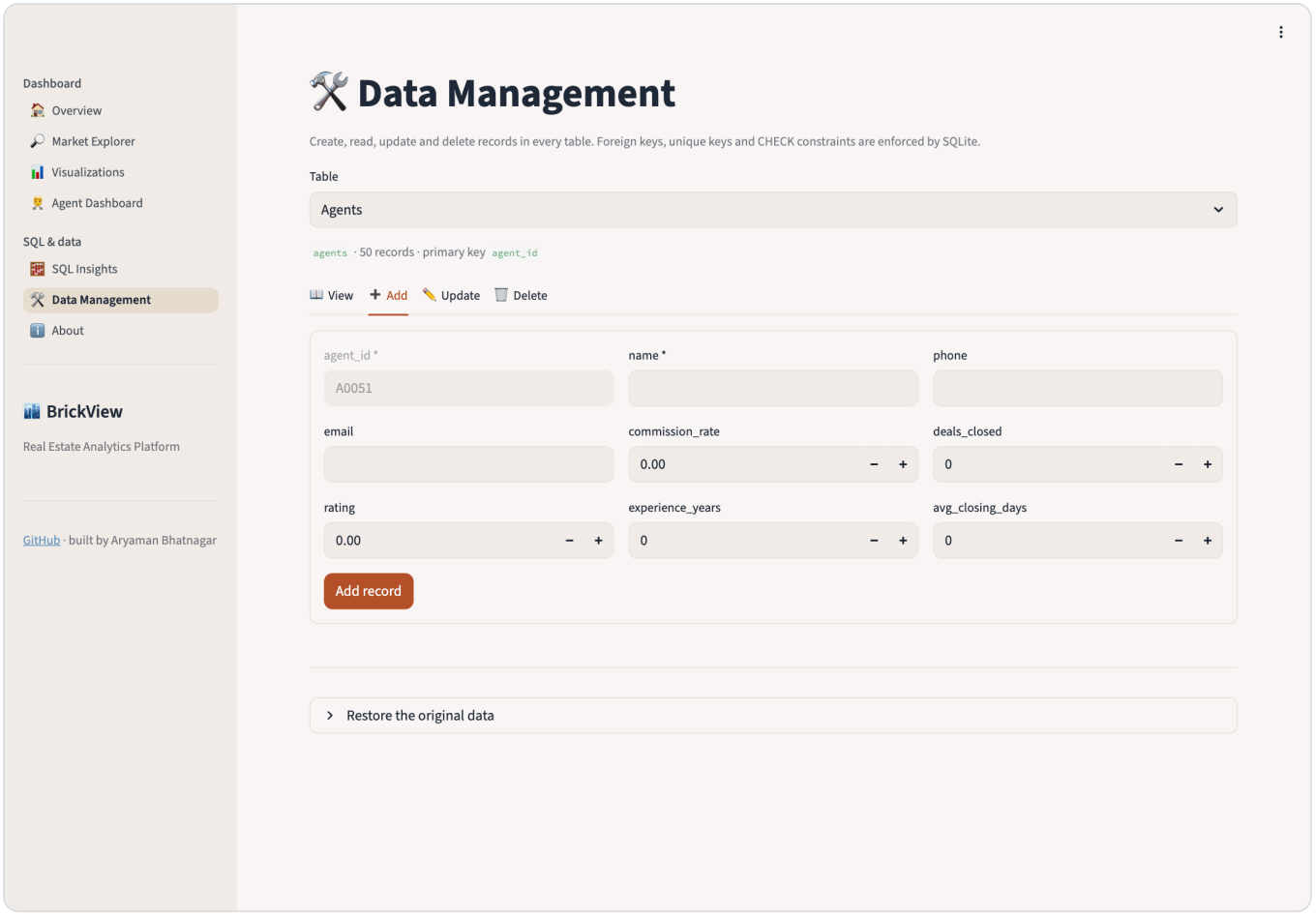
Buyer and financing behaviour.



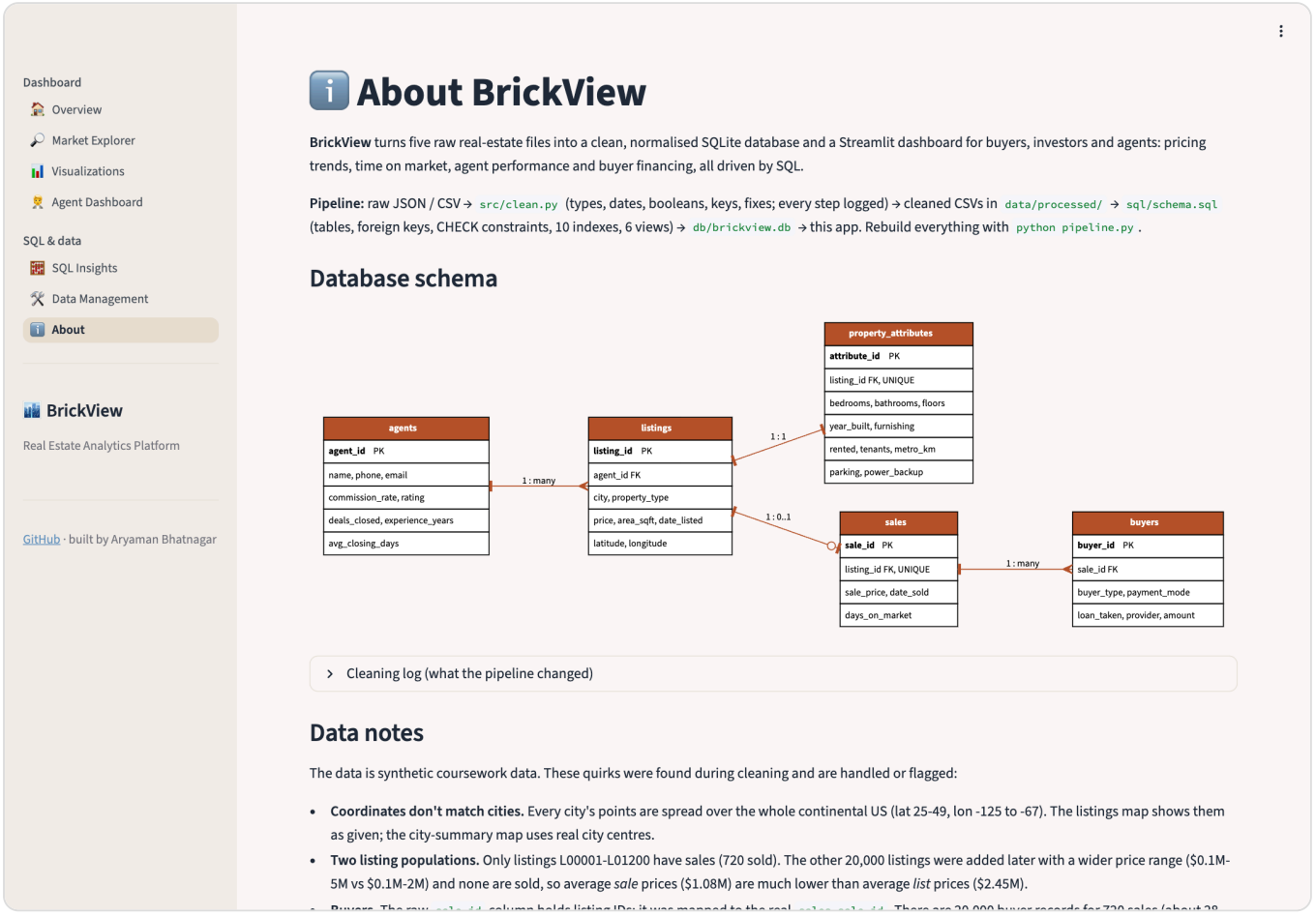
Agent dashboard: leaderboard, experience and rating vs performance, per-agent profile.



SQL Insights: every question in a drop-down with its SQL, live output, chart and finding.



Data Management: create, read, update and delete for all five tables, with constraint checks.



About: schema diagram, cleaning log and data notes.

5. SQL analysis: 30 business questions

Property & Pricing

Q01. What is the average listing price by city?

```
SELECT city,
       COUNT(*)              AS listings,
       ROUND(AVG(price), 2)  AS avg_list_price
FROM listings
GROUP BY city
ORDER BY avg_list_price DESC
```

city	listings	avg_list_price
New York	4,269	$2,493,319.74
Phoenix	4,229	$2,459,961.76
Los Angeles	4,318	$2,442,418.31
Houston	4,091	$2,436,811.00
Chicago	4,293	$2,430,677.90

Answer: New York is the priciest city ($2.49M average) and Chicago the cheapest ($2.43M), a spread of only 2.6%.

Q02. What is the average price per square foot by property type?

```
SELECT property_type,
       COUNT(*)                                 AS listings,
       ROUND(AVG(price / area_sqft), 2)         AS avg_price_per_sqft,
       ROUND(SUM(price) / SUM(area_sqft), 2)    AS weighted_price_per_sqft
FROM listings
GROUP BY property_type
ORDER BY avg_price_per_sqft DESC
```

property_type	listings	avg_price_per_sqft	weighted_price_per_sqft
House	5,330	$796.04	$484.31
Apartment	5,335	$792.20	$479.84
Townhouse	5,172	$789.75	$484.10
Condo	5,363	$754.68	$470.73

Answer: Houses have the highest price per sq ft ($796) and condos the lowest ($755). The plain average is far above the size-weighted figure (~$480) because small units carry very high per-sq-ft prices.

Q03. How does furnishing status impact property prices?

```
SELECT pa.furnishing_status,
       COUNT(*)                                AS listings,
       ROUND(AVG(l.price), 2)                  AS avg_list_price,
       ROUND(AVG(l.price / l.area_sqft), 2)    AS avg_price_per_sqft
FROM listings l
JOIN property_attributes pa ON pa.listing_id = l.listing_id
GROUP BY pa.furnishing_status
ORDER BY avg_list_price DESC
```

furnishing_status	listings	avg_list_price	avg_price_per_sqft
Furnished	6,977	$2,463,539.06	$792.41
Semi-Furnished	7,013	$2,461,192.65	$780.41
Unfurnished	7,210	$2,433,975.10	$776.64

> **Answer:** Furnished homes list about 1.2% higher than unfurnished ones: a small premium.

Q04. Do properties closer to metro stations command higher prices?

```
SELECT CASE WHEN pa.metro_distance_km < 2  THEN '1. Under 2 km'
            WHEN pa.metro_distance_km < 5  THEN '2. 2-5 km'
            WHEN pa.metro_distance_km < 10 THEN '3. 5-10 km'
            ELSE '4. 10 km+' END               AS metro_distance,
       COUNT(*)                                AS listings,
       ROUND(AVG(l.price), 2)                  AS avg_list_price,
       ROUND(AVG(l.price / l.area_sqft), 2)    AS avg_price_per_sqft
FROM listings l
JOIN property_attributes pa ON pa.listing_id = l.listing_id
GROUP BY metro_distance
ORDER BY metro_distance
```

metro_distance	listings	avg_list_price	avg_price_per_sqft
1. Under 2 km	2,888	$2,376,124.22	$763.46
2. 2-5 km	4,419	$2,407,369.68	$763.84
3. 5-10 km	7,221	$2,442,765.21	$787.27
4. 10 km+	6,672	$2,526,648.02	$799.76

> **Answer:** No metro premium. Listings within 2 km of a metro average $2.38M, those 10 km+ away $2.53M. Distance does not raise prices in this data.

Q05. Are rented properties priced differently from non-rented ones?

```
SELECT CASE pa.is_rented WHEN 1 THEN 'Rented' ELSE 'Not rented' END AS rental_status,
       COUNT(*)                                  AS listings,
       ROUND(AVG(l.price), 2)                    AS avg_list_price,
       ROUND(AVG(l.price / l.area_sqft), 2)      AS avg_price_per_sqft
FROM listings l
JOIN property_attributes pa ON pa.listing_id = l.listing_id
GROUP BY pa.is_rented
ORDER BY avg_list_price DESC
```

rental_status	listings	avg_list_price	avg_price_per_sqft
Rented	10,578	$2,457,184.90	$789.36
Not rented	10,622	$2,448,250.30	$776.81

> **Answer:** Rented and non-rented properties are priced almost the same (+0.4% for rented).

Q06. How do bedrooms and bathrooms affect pricing?

```
SELECT pa.bedrooms,
       pa.bathrooms,
       COUNT(*)                 AS listings,
       ROUND(AVG(l.price), 2)   AS avg_list_price
FROM listings l
JOIN property_attributes pa ON pa.listing_id = l.listing_id
GROUP BY pa.bedrooms, pa.bathrooms
ORDER BY pa.bedrooms, pa.bathrooms
```

bedrooms	bathrooms	listings	avg_list_price
1	1	1,040	$2,434,001.41
1	2	1,047	$2,509,474.77
1	3	1,028	$2,459,032.59
1	4	1,094	$2,455,532.96
2	1	1,068	$2,455,735.36
2	2	1,127	$2,378,687.66
2	3	1,060	$2,482,618.93
2	4	1,021	$2,555,766.23
3	1	1,032	$2,426,007.97
3	2	1,064	$2,409,122.73

Showing 10 of 20 rows.

> **Answer:** Bedroom and bathroom counts barely move the average price (every group sits between about $2.38M and $2.56M): size is not priced in this data set.

Q07. Do properties with parking and power backup sell at higher prices?

```
SELECT CASE WHEN pa.parking_available = 1 AND pa.power_backup = 1 THEN 'Parking + power backup'
            WHEN pa.parking_available = 1 THEN 'Parking only'
            WHEN pa.power_backup = 1 THEN 'Power backup only'
            ELSE 'Neither' END                  AS amenities,
       COUNT(*)                                 AS listings,
       ROUND(AVG(l.price), 2)                   AS avg_list_price,
       COUNT(s.sale_id)                         AS sales,
       ROUND(AVG(s.sale_price), 2)              AS avg_sale_price
FROM listings l
JOIN property_attributes pa ON pa.listing_id = l.listing_id
LEFT JOIN sales s           ON s.listing_id  = l.listing_id
GROUP BY amenities
ORDER BY avg_sale_price DESC
```

amenities	listings	avg_list_price	sales	avg_sale_price
Parking + power backup	5,243	$2,465,651.62	170	$1,092,597.11
Parking only	5,383	$2,462,363.80	198	$1,085,159.68
Power backup only	5,166	$2,446,292.59	170	$1,085,149.05
Neither	5,408	$2,436,677.75	182	$1,048,102.17

Answer: Homes with both parking and power backup sold for the most ($1.09M average), about 4% more than homes with neither ($1.05M).

Q08. How does year built influence listing price?

```
SELECT pa.year_built,
       COUNT(*)                 AS listings,
       ROUND(AVG(l.price), 2)   AS avg_list_price
FROM listings l
JOIN property_attributes pa ON pa.listing_id = l.listing_id
GROUP BY pa.year_built
ORDER BY pa.year_built
```

year_built	listings	avg_list_price
1990	607	$2,533,791.60
1991	605	$2,573,916.58
1992	574	$2,594,651.56
1993	548	$2,611,748.25
1994	608	$2,571,504.92
1995	608	$2,434,727.72
1996	617	$2,391,385.75
1997	610	$2,493,105.20
1998	649	$2,398,158.87
1999	655	$2,498,402.48

Showing 10 of 34 rows.

Answer: No premium for new construction: homes built 1990-1994 list highest ($2.58M average), later 5-year cohorts sit between $2.37M and $2.48M.

Q09. Which cities have the highest average property prices?

```
WITH ordered AS (
    SELECT city, price,
           ROW_NUMBER() OVER (PARTITION BY city ORDER BY price) AS rn,
           COUNT(*)     OVER (PARTITION BY city)                AS n
    FROM listings
),
median AS (
    SELECT city, ROUND(AVG(price), 2) AS median_list_price
    FROM ordered
    WHERE rn IN ((n + 1) / 2, (n + 2) / 2)
    GROUP BY city
)
SELECT RANK() OVER (ORDER BY AVG(l.price) DESC) AS price_rank,
       l.city,
       ROUND(AVG(l.price), 2)  AS avg_list_price,
       m.median_list_price,
       ROUND(MAX(l.price), 2)  AS max_list_price
FROM listings l
JOIN median m ON m.city = l.city
GROUP BY l.city
ORDER BY price_rank
```

price_rank	city	avg_list_price	median_list_price	max_list_price
1	New York	$2,493,319.74	$2,446,518.63	$4,998,508.49
2	Phoenix	$2,459,961.76	$2,397,597.97	$4,999,863.52
3	Los Angeles	$2,442,418.31	$2,380,929.35	$4,999,463.77
4	Houston	$2,436,811.00	$2,347,582.06	$4,998,687.21
5	Chicago	$2,430,677.90	$2,351,084.47	$4,997,593.10

Answer: New York ranks first on both average ($2.49M) and median ($2.45M) price; the ranking is the same on either measure.

Q10. How are properties distributed across price buckets?

```
SELECT CASE WHEN price < 1000000 THEN '1. Under $1M'
            WHEN price < 2000000 THEN '2. $1M-2M'
            WHEN price < 3000000 THEN '3. $2M-3M'
            WHEN price < 4000000 THEN '4. $3M-4M'
            ELSE '5. $4M+' END                                        AS price_bucket,
       COUNT(*)                                                        AS listings,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM listings), 2) AS pct_of_listings
FROM listings
GROUP BY price_bucket
ORDER BY price_bucket
```

price_bucket	listings	pct_of_listings
1. Under $1M	4,269	20.14
2. $1M-2M	4,800	22.64
3. $2M-3M	4,025	18.99
4. $3M-4M	4,083	19.26
5. $4M+	4,023	18.98

> **Answer:** Listings are spread almost evenly across the five $1M-wide buckets (19-23% each); $1M-2M is the largest.

Sales & Market

Q11. What is the average days on market by city?

```
SELECT l.city,
       COUNT(*)                          AS sales,
       ROUND(AVG(s.days_on_market), 1)   AS avg_days_on_market
FROM sales s
JOIN listings l ON l.listing_id = s.listing_id
GROUP BY l.city
ORDER BY avg_days_on_market
```

city	sales	avg_days_on_market
Houston	131	58.50
Phoenix	168	59.70
New York	137	60.80
Chicago	156	64.30
Los Angeles	128	65.10

> **Answer:** Houston sells fastest (58.5 days on average) and Los Angeles slowest (65.1 days).

Q12. Which property types sell the fastest?

```
SELECT l.property_type,
       COUNT(*)                          AS sales,
       ROUND(AVG(s.days_on_market), 1)   AS avg_days_on_market,
       MIN(s.days_on_market)             AS fastest_sale_days
FROM sales s
JOIN listings l ON l.listing_id = s.listing_id
GROUP BY l.property_type
ORDER BY avg_days_on_market
```

property_type	sales	avg_days_on_market	fastest_sale_days
House	190	58.30	5
Apartment	176	60.60	5
Townhouse	167	61.00	5
Condo	187	66.50	5

> **Answer:** Houses sell fastest (58.3 days); condos take the longest (66.5 days), about 8 days more.

Q13. What percentage of properties are sold above listing price?

```
SELECT CASE WHEN s.sale_price > l.price THEN 'Above list price'
            WHEN s.sale_price = l.price THEN 'At list price'
            ELSE 'Below list price' END                          AS outcome,
       COUNT(*)                                                   AS sales,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM sales), 2)  AS pct_of_sales
FROM sales s
JOIN listings l ON l.listing_id = s.listing_id
GROUP BY outcome
ORDER BY sales DESC
```

outcome	sales	pct_of_sales
Below list price	365	50.69
Above list price	355	49.31

> **Answer:** 49.3% of sales closed above the listing price and 50.7% below: pricing is close to fair value.

Q14. What is the sale-to-list price ratio by city?

```
SELECT l.city,
       COUNT(*)                                    AS sales,
       ROUND(AVG(s.sale_price / l.price), 4)       AS avg_sale_to_list_ratio,
       ROUND(SUM(s.sale_price) / SUM(l.price), 4)  AS total_sale_to_list_ratio
FROM sales s
JOIN listings l ON l.listing_id = s.listing_id
GROUP BY l.city
ORDER BY avg_sale_to_list_ratio DESC
```

city	sales	avg_sale_to_list_ratio	total_sale_to_list_ratio
Chicago	156	1.0015	1.0004
Houston	131	1.0000	1.0003
Los Angeles	128	0.9998	0.9981
New York	137	0.9996	0.9981
Phoenix	168	0.9989	0.9985

> **Answer:** Sale-to-list ratios sit within 0.2% of 1.00 in every city; Chicago is the only city slightly above list (1.0015).

Q15. Which listings took more than 90 days to sell?

```
SELECT s.listing_id, l.city, l.property_type, l.price AS list_price, s.sale_price,
       l.date_listed, s.date_sold, s.days_on_market
FROM sales s
JOIN listings l ON l.listing_id = s.listing_id
WHERE s.days_on_market > 90
ORDER BY s.days_on_market DESC, s.listing_id
```

listing_id	city	property_type	list_price	sale_price	date_listed	date_sold	days_on_market
L00067	Chicago	Apartment	$293,562.99	$286,681.00	2024-02-29	2024-06-28	120
L00157	Phoenix	Townhouse	$710,815.00	$707,140.98	2023-04-29	2023-08-27	120
L00179	Chicago	Condo	$971,345.99	$1,017,871.01	2023-08-30	2023-12-28	120
L00259	Phoenix	Apartment	$1,513,413.99	$1,577,127.00	2023-05-20	2023-09-17	120
L00287	Houston	Townhouse	$1,724,170.00	$1,769,308.00	2023-12-14	2024-04-12	120
L00057	Phoenix	Condo	$646,864.99	$665,657.00	2023-12-09	2024-04-06	119
L00078	Los Angeles	House	$149,627.00	$152,524.01	2023-07-24	2023-11-20	119
L00271	New York	House	$1,013,679.01	$970,384.01	2023-09-26	2024-01-23	119
L00348	New York	Condo	$1,969,257.99	$2,031,708.00	2024-03-11	2024-07-08	119
L00351	Phoenix	Condo	$1,080,221.01	$1,032,111.00	2023-11-10	2024-03-08	119

Showing 10 of 188 rows.

> **Answer:** 188 of 720 sales (26%) took more than 90 days; the slowest took 120 days.

Q16. How does metro distance affect time on market?

```
SELECT CASE WHEN pa.metro_distance_km < 2  THEN '1. Under 2 km'
            WHEN pa.metro_distance_km < 5  THEN '2. 2-5 km'
            WHEN pa.metro_distance_km < 10 THEN '3. 5-10 km'
            ELSE '4. 10 km+' END            AS metro_distance,
       COUNT(*)                             AS sales,
       ROUND(AVG(s.days_on_market), 1)      AS avg_days_on_market
FROM sales s
JOIN property_attributes pa ON pa.listing_id = s.listing_id
GROUP BY metro_distance
ORDER BY metro_distance
```

metro_distance	sales	avg_days_on_market
1. Under 2 km	153	64.70
2. 2-5 km	265	60.90
3. 5-10 km	302	60.70

Answer: Homes within 2 km of a metro took the longest to sell (64.7 days vs about 61 elsewhere); no sold home is more than 8 km from a metro.

Q17. What is the monthly sales trend?

```
SELECT strftime('%Y-%m', date_sold)   AS month,
       COUNT(*)                       AS sales,
       ROUND(SUM(sale_price), 2)      AS revenue,
       ROUND(AVG(sale_price), 2)      AS avg_sale_price
FROM sales
GROUP BY month
ORDER BY month
```

month	sales	revenue	avg_sale_price
2023-01	7	$5,298,400.96	$756,914.42
2023-02	17	$15,945,070.02	$937,945.30
2023-03	20	$19,995,066.06	$999,753.30
2023-04	36	$44,208,249.99	$1,228,006.94
2023-05	45	$51,385,673.97	$1,141,903.87
2023-06	31	$29,643,443.03	$956,240.10
2023-07	48	$56,000,209.96	$1,166,671.04
2023-08	50	$55,050,662.07	$1,101,013.24
2023-09	46	$49,868,704.96	$1,084,102.28
2023-10	64	$70,202,528.99	$1,096,914.52

Showing 10 of 21 rows.

Answer: Sales ramp up through 2023, peak in October 2023 (64 sales), and taper off after mid-2024.

Q18. Which properties are currently unsold?

```
SELECT l.listing_id, l.city, l.property_type, l.price, l.area_sqft, l.date_listed, l.agent_id
FROM listings l
WHERE NOT EXISTS (SELECT 1 FROM sales s WHERE s.listing_id = l.listing_id)
ORDER BY l.date_listed, l.listing_id
```

listing_id	city	property_type	price	area_sqft	date_listed	agent_id
L01460	Los Angeles	Townhouse	$3,049,053.39	3,416.27	2023-01-01	A0034
L02158	Chicago	Condo	$3,447,822.35	5,391.60	2023-01-01	A0035
L04325	New York	Apartment	$737,399.70	6,265.78	2023-01-01	A0014
L04378	Phoenix	Apartment	$3,976,515.39	5,091.39	2023-01-01	A0013
L07239	Los Angeles	Townhouse	$3,611,871.66	7,323.45	2023-01-01	A0043
L07425	Chicago	House	$3,325,174.11	9,842.94	2023-01-01	A0041
L08024	Phoenix	Condo	$3,305,636.54	4,479.84	2023-01-01	A0045
L08311	New York	Apartment	$2,713,607.49	9,060.33	2023-01-01	A0013
L08520	Chicago	Condo	$4,070,408.07	6,273.11	2023-01-01	A0011
L08677	Chicago	Condo	$1,670,092.09	3,404.30	2023-01-01	A0012

Showing 10 of 20,480 rows.

> **Answer:** 20,480 of 21,200 listings (96.6%) are still unsold. All 720 sales come from the first 1,200 listings (L00001-L01200).

Agent Performance

Q19. Which agents have closed the most sales?

```
SELECT a.agent_id, a.name,
       COUNT(s.sale_id) AS sales_closed
FROM agents a
JOIN listings l ON l.agent_id   = a.agent_id
JOIN sales s    ON s.listing_id = l.listing_id
GROUP BY a.agent_id
ORDER BY sales_closed DESC, a.agent_id
LIMIT 10
```

agent_id	name	sales_closed
A0042	Agent A0042	25
A0011	Agent A0011	24
A0014	Agent A0014	21
A0035	Agent A0035	21
A0043	Agent A0043	20
A0046	Agent A0046	20
A0007	Agent A0007	19
A0048	Agent A0048	19
A0027	Agent A0027	18
A0029	Agent A0029	18

> **Answer:** Agent A0042 closed the most sales (25), followed by A0011 (24).

Q20. Who are the top agents by total sales revenue?

```
SELECT a.agent_id, a.name,
       COUNT(s.sale_id)              AS sales_closed,
       ROUND(SUM(s.sale_price), 2)   AS total_revenue
FROM agents a
JOIN listings l ON l.agent_id   = a.agent_id
JOIN sales s    ON s.listing_id = l.listing_id
GROUP BY a.agent_id
ORDER BY total_revenue DESC
LIMIT 10
```

agent_id	name	sales_closed	total_revenue
A0011	Agent A0011	24	$27,882,272.01
A0042	Agent A0042	25	$27,191,605.99
A0043	Agent A0043	20	$24,102,418.01
A0035	Agent A0035	21	$22,725,751.98
A0014	Agent A0014	21	$22,034,008.02
A0046	Agent A0046	20	$21,335,804.97
A0048	Agent A0048	19	$21,186,305.99
A0027	Agent A0027	18	$21,099,696.97
A0009	Agent A0009	17	$20,279,274.94
A0029	Agent A0029	18	$19,586,498.01

Answer: Agent A0011 generated the most revenue ($27.9M from 24 sales), just ahead of A0042 ($27.2M).

Q21. Which agents close deals fastest?

```
SELECT a.agent_id, a.name,
       COUNT(s.sale_id)                  AS sales_closed,
       ROUND(AVG(s.days_on_market), 1)   AS avg_days_on_market,
       a.avg_closing_days                AS profile_avg_closing_days
FROM agents a
JOIN listings l ON l.agent_id   = a.agent_id
JOIN sales s    ON s.listing_id = l.listing_id
GROUP BY a.agent_id
HAVING COUNT(s.sale_id) >= 5
ORDER BY avg_days_on_market, a.agent_id
LIMIT 10
```

agent_id	name	sales_closed	avg_days_on_market	profile_avg_closing_days
A0044	Agent A0044	8	36.90	35
A0013	Agent A0013	7	42.40	65
A0037	Agent A0037	12	44.50	77
A0002	Agent A0002	8	45.50	82
A0008	Agent A0008	11	46.60	72
A0017	Agent A0017	14	48.60	58
A0026	Agent A0026	12	49.50	42
A0012	Agent A0012	11	50.60	82
A0010	Agent A0010	15	51.10	90
A0043	Agent A0043	20	52.40	89

> **Answer:** Among agents with at least 5 sales, A0044 is fastest (36.9 days on market on average).

Q22. Does experience correlate with deals closed?

```
SELECT CASE WHEN experience_years <= 5  THEN '1. 0-5 yrs'
            WHEN experience_years <= 10 THEN '2. 6-10 yrs'
            WHEN experience_years <= 15 THEN '3. 11-15 yrs'
            WHEN experience_years <= 20 THEN '4. 16-20 yrs'
            ELSE '5. 21+ yrs' END               AS experience,
       COUNT(*)                                 AS agents,
       ROUND(AVG(deals_closed_lifetime), 1)     AS avg_deals_closed,
       ROUND(AVG(sales), 1)                     AS avg_sales_in_dataset
FROM v_agent_performance
GROUP BY experience
ORDER BY experience
```

experience	agents	avg_deals_closed	avg_sales_in_dataset
1. 0-5 yrs	8	108.90	16.10
2. 6-10 yrs	5	123.20	14.20
3. 11-15 yrs	10	141.10	13.50
4. 16-20 yrs	17	133.50	12.70
5. 21+ yrs	10	166.40	16.90

> **Answer:** Weak positive link: agents with 21+ years average 166 lifetime deals vs 109 for 0-5 years, but the trend is not steady and sales in this data set don't follow it.

Q23. Do agents with higher ratings close deals faster?

```
SELECT CASE WHEN rating < 3.5 THEN '1. Below 3.5'
            WHEN rating < 4.0 THEN '2. 3.5–3.9'
            WHEN rating < 4.5 THEN '3. 4.0–4.4'
            ELSE '4. 4.5–5.0' END                AS rating_band,
       COUNT(*)                                  AS agents,
       ROUND(AVG(avg_closing_days), 1)           AS avg_closing_days,
       ROUND(AVG(avg_days_on_market), 1)         AS avg_days_on_market_in_dataset
FROM v_agent_performance
GROUP BY rating_band
ORDER BY rating_band
```

rating_band	agents	avg_closing_days	avg_days_on_market_in_dataset
1. Below 3.5	12	56.80	60.10
2. 3.5-3.9	12	67.70	60.40
3. 4.0-4.4	16	48.40	60.30
4. 4.5-5.0	10	51.70	62.40

Answer: No: the 4.5-5.0 rated agents take about as long (51.7 days) as the 4.0-4.4 band (48.4); the slowest band is 3.5-3.9 (67.7).

Q24. What is the average commission earned by each agent?

```
SELECT a.agent_id, a.name, a.commission_rate,
       COUNT(s.sale_id)                                          AS sales_closed,
       ROUND(AVG(s.sale_price * a.commission_rate / 100.0), 2)   AS avg_commission_per_sale,
       ROUND(SUM(s.sale_price * a.commission_rate / 100.0), 2)   AS total_commission
FROM agents a
JOIN listings l ON l.agent_id   = a.agent_id
JOIN sales s    ON s.listing_id = l.listing_id
GROUP BY a.agent_id
ORDER BY avg_commission_per_sale DESC
```

agent_id	name	commission_rate	sales_closed	avg_commission_per_sale	total_commission
A0009	Agent A0009	$2.85	17	$33,997.61	$577,959.34
A0002	Agent A0002	$2.20	8	$32,326.16	$258,609.25
A0035	Agent A0035	$2.96	21	$32,032.49	$672,682.26
A0012	Agent A0012	$2.37	11	$31,737.61	$349,113.68
A0030	Agent A0030	$2.79	11	$31,665.09	$348,316.02
A0018	Agent A0018	$2.63	13	$31,438.36	$408,698.74
A0031	Agent A0031	$2.51	13	$30,358.05	$394,654.65
A0050	Agent A0050	$2.56	17	$28,564.40	$485,594.83
A0017	Agent A0017	$2.14	14	$28,406.74	$397,694.41
A0024	Agent A0024	$2.95	14	$27,805.90	$389,282.53

Showing 10 of 50 rows.

Answer: Average commission per sale ranges up to $34.0k (A0009, at a 2.85% rate). Total commission is driven mostly by sales volume.

Q25. Which agents currently have the most active listings?

```
SELECT a.agent_id, a.name,
       COUNT(*) AS active_listings
FROM agents a
JOIN v_active_listings al ON al.agent_id = a.agent_id
GROUP BY a.agent_id
ORDER BY active_listings DESC, a.agent_id
LIMIT 10
```

agent_id	name	active_listings
A0023	Agent A0023	446
A0011	Agent A0011	439
A0008	Agent A0008	438
A0042	Agent A0042	435
A0014	Agent A0014	432
A0044	Agent A0044	430
A0020	Agent A0020	426
A0012	Agent A0012	425
A0015	Agent A0015	425
A0048	Agent A0048	425

> **Answer:** Agent A0023 has the most active listings (446); the top 10 are all within 446-425.

Buyer & Financing

Q26. What percentage of buyers are investors vs end users?

```
SELECT buyer_type,
       COUNT(*)                                                    AS buyers,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM buyers), 2)  AS pct_of_buyers
FROM buyers
GROUP BY buyer_type
ORDER BY buyers DESC
```

buyer_type	buyers	pct_of_buyers
End User	10,020	50.10
Investor	9,980	49.90

> **Answer:** Buyers split evenly: 50.1% end users and 49.9% investors.

Q27. Which cities have the highest loan uptake rate?

```
SELECT l.city,
       COUNT(*)                          AS buyers,
       SUM(b.loan_taken)                 AS buyers_with_loan,
       ROUND(AVG(b.loan_taken) * 100, 2)   AS loan_uptake_pct
FROM buyers b
JOIN sales s    ON s.sale_id    = b.sale_id
JOIN listings l ON l.listing_id = s.listing_id
GROUP BY l.city
ORDER BY loan_uptake_pct DESC
```

city	buyers	buyers_with_loan	loan_uptake_pct
Los Angeles	3,552	1,811	50.99
Chicago	4,327	2,188	50.57
New York	3,809	1,913	50.22
Houston	3,639	1,809	49.71
Phoenix	4,673	2,313	49.50

Answer: Loan uptake is about 50% everywhere, from 49.5% (Phoenix) to 51.0% (Los Angeles).

Q28. What is the average loan amount by buyer type?

```
SELECT buyer_type,
       COUNT(*)                      AS loans,
       ROUND(AVG(loan_amount), 2)    AS avg_loan_amount
FROM buyers
WHERE loan_taken = 1
GROUP BY buyer_type
ORDER BY avg_loan_amount DESC
```

buyer_type	loans	avg_loan_amount
Investor	5,035	$5,211,545.97
End User	4,999	$5,210,307.45

Answer: Investors and end users borrow almost the same on average (about $5.21M).

Q29. Which payment mode is most commonly used?

```
SELECT payment_mode,
       COUNT(*)                                                    AS buyers,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM buyers), 2)   AS pct_of_buyers
FROM buyers
GROUP BY payment_mode
ORDER BY buyers DESC
```

payment_mode	buyers	pct_of_buyers
Cash	5,088	25.44
UPI	5,012	25.06
Cheque	4,951	24.75
Bank Transfer	4,949	24.75

> **Answer:** Cash is used most (25.4%), but all four payment modes are within 0.7 points of each other.

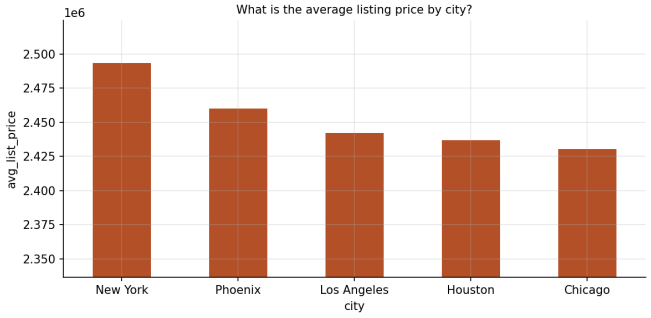
Q30. Do loan-backed purchases take longer to close?

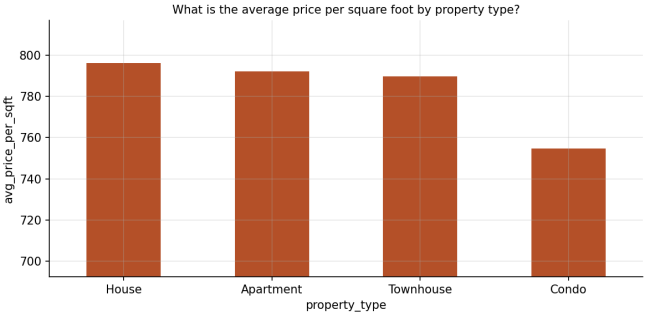
```
SELECT CASE b.loan_taken WHEN 1 THEN 'Loan-backed' ELSE 'No loan' END AS financing,
       COUNT(*)                            AS buyers,
       ROUND(AVG(s.days_on_market), 2)     AS avg_days_on_market
FROM buyers b
JOIN sales s ON s.sale_id = b.sale_id
GROUP BY b.loan_taken
ORDER BY avg_days_on_market DESC
```

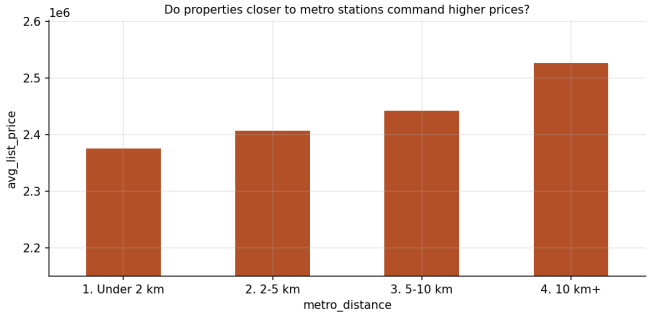
financing	buyers	avg_days_on_market
Loan-backed	10,034	62.15
No loan	9,966	61.07

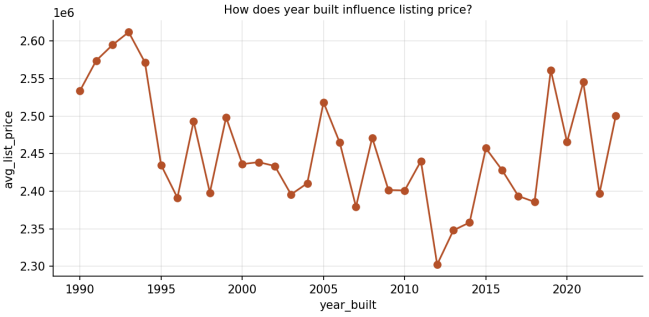
> **Answer:** Loan-backed purchases took about one day longer (62.2 vs 61.1 days on market): a negligible difference.

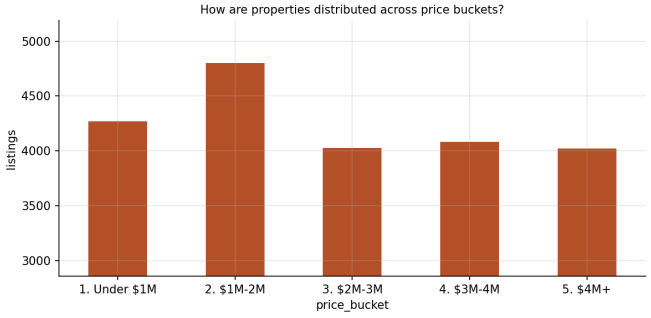
6. Selected charts

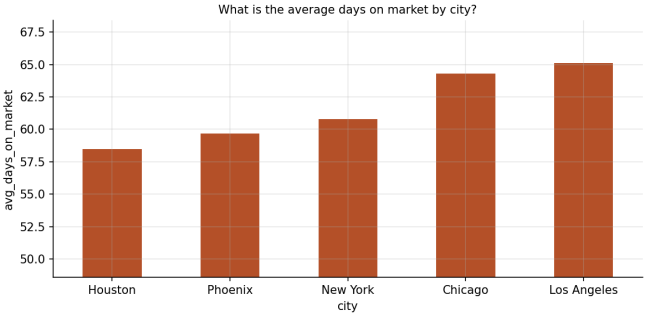




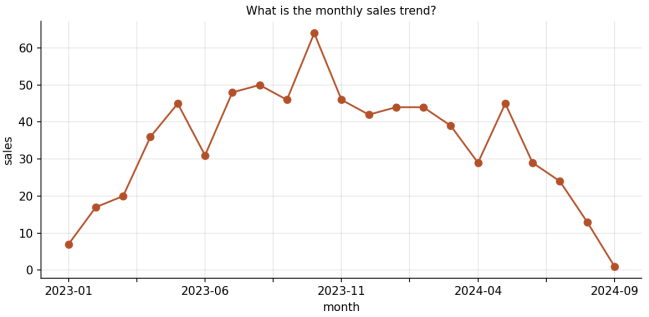


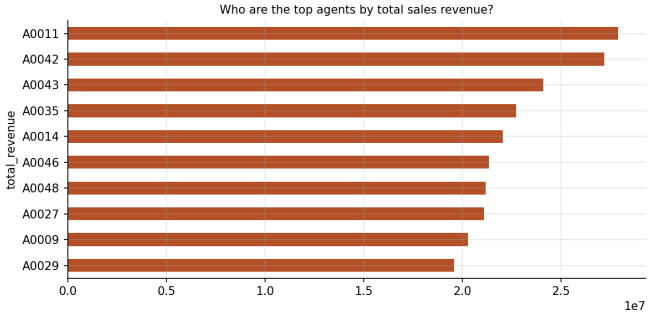


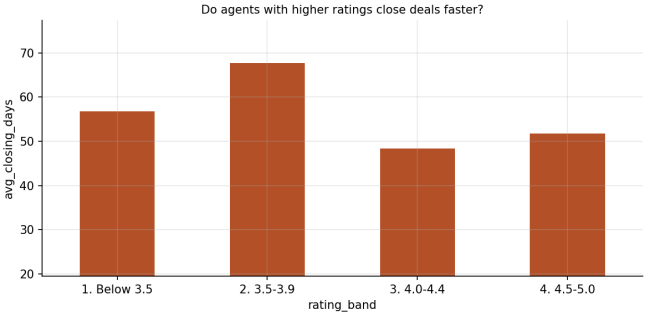


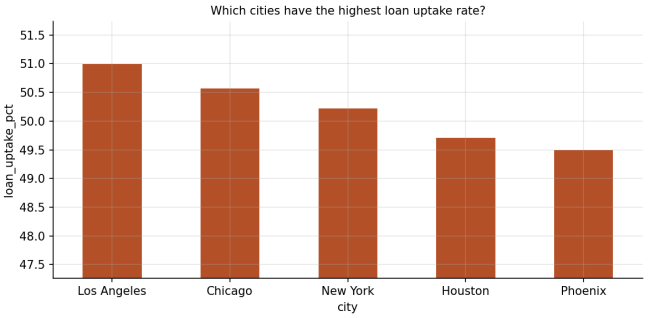


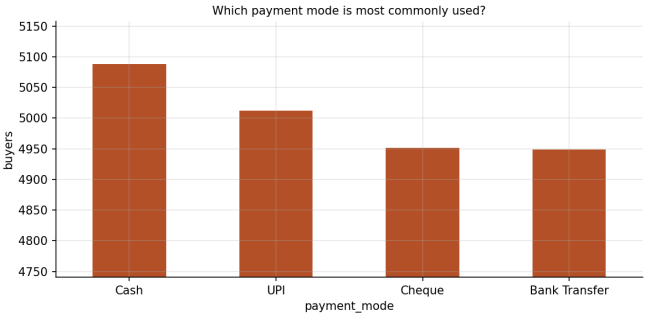












7. Key insights and recommendations

1. **Price to the market, not above it:** sale-to-list is within 0.2% of 1.00 everywhere and half of homes sell below list.
2. **New York commands the highest prices** ($2.49M average), but the city spread is only 2.6%.
3. **Houses move fastest** (58 days) and condos slowest (67); Houston is the fastest market, Los Angeles the slowest. Agents should expect longer campaigns for condos in LA and Chicago.
4. **Amenities and location add little:** no metro premium, about 1% for furnishing, about 4% on sale price for parking + power backup.
5. **Demand peaked in October 2023** and has slowed since; 26% of sales took over 90 days, a pool worth targeting with price reviews.
6. **Ratings don't predict speed** and experience is only weakly linked to deals, so agent assignment should use measured days-on-market and revenue (Agent Dashboard) rather than profile ratings.
7. **Buyers are balanced:** 50/50 investors and end users, about half use a loan, and all payment modes are used equally.
8. **Data caveat:** the dataset is synthetic (uniform attributes, random coordinates, unrealistic loan sizes), so effects are small and should be re-checked on real market data.

8. Deliverables

Cleaned CSV data	`data/processed/*.csv` + `cleaning_log.csv`
SQL schema	`sql/schema.sql` (source) and `sql/schema_dump.sql` (as stored)
30 SQL queries	`sql/queries.sql`
Analysis notebook	`notebooks/eda.ipynb`
Streamlit app	`app/` · https://brickview-real-estate.streamlit.app
Tests	`tests/test_brickview.py` (43 checks)
Report and screenshots	`reports/`